THREAT TO RISK

From Theory to Practice: Protecting Digital Assets Using Real-World Security Frameworks

Learning Objectives

By the end, we should be able to:

1. Explain the difference between **threat**, **vulnerability**, **asset**, **impact**, and **risk**
2. Build complete risk scenarios from a **real-world system**
3. Apply the idea to a simulation diagrams with:
   SQL injection
   plaintext passwords
   missing MFA
4. Explain how scenarios improve **risk analysis accuracy**

Real-World System Example

The portal contains:

Student accounts

Admin accounts

Grades

Login page

SQLite database

Search page

Why this system important to us?:

If the portal is compromised, the result is not just “a technical bug.” It will effect student privacy, grades, integrity, university reputation, and trust.

Key Terms

Asset: The "Gold." This is your data, your server, or your reputation.

Vulnerability: The "Open Window." A bug in code or a misconfigured server.

Threat: The "Burglar." The actor (hacker, insider, bot) attempting to intrude.

Risk: The "Break-in." The chance the burglar succeeds and causes actual loss.

Impact: The “Fire“, The damage that attack will leave in our assets.

Building Correct Risk Scenario

A strong risk scenario must be:

- **Specific:** names the exact asset
- **Realistic:** uses a believable threat
- **Complete:** includes asset, threat, vulnerability, and impact
- **Measurable:** includes likelihood and risk level
- **Actionable:** connects to a control or fix

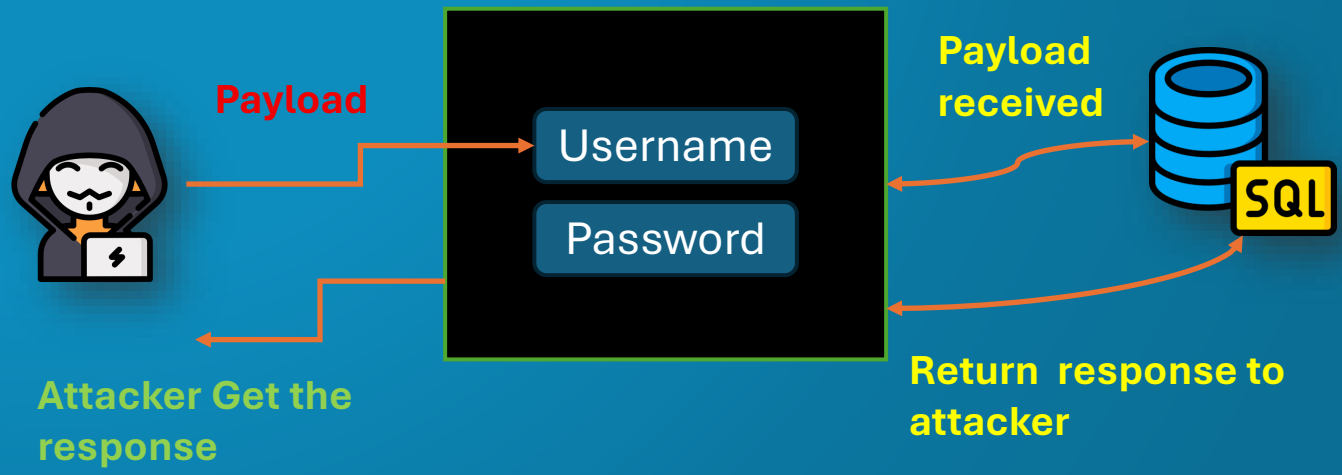
Weak scenario:

"**SQL injection** is dangerous."

Strong scenario:

"An attacker exploits **SQL injection** in the student portal login,

because the app builds SQL queries using user input, causing unauthorized access to student accounts."



THEAT VS VULNERABILITY

THEAT IS Vulnerability = Not Correct

Threat = who or what causes harm

Vulnerability IS THREAT = Not Correct

Vulnerability = the weakness that makes harm possible

Incorrect

“SQL injection is the threat.”

“The hacker is the vulnerability.”

“Plaintext passwords are the threat.”

Correct

SQL injection is the vulnerability.

The hacker is the threat actor.

Plaintext passwords are the vulnerability.

Simulation

Vulnerable SQLite Portal

Vulnerable design:

1) SQLite database stores users and grades
2) Login query is vulnerable to SQL injection
3) Passwords are stored in plaintext
4) Admin users do not use MFA

Scenario 1: SQL Injection Login Bypass

Asset:	**Student portal login and user accounts**
Threat:	**Attacker attempts unauthorized login**
Vulnerability:	**Login query uses unsafe dynamic SQL**
Impact:	**Unauthorized access to student or admin account**
Likelihood:	**High**
Risk level:	**High**

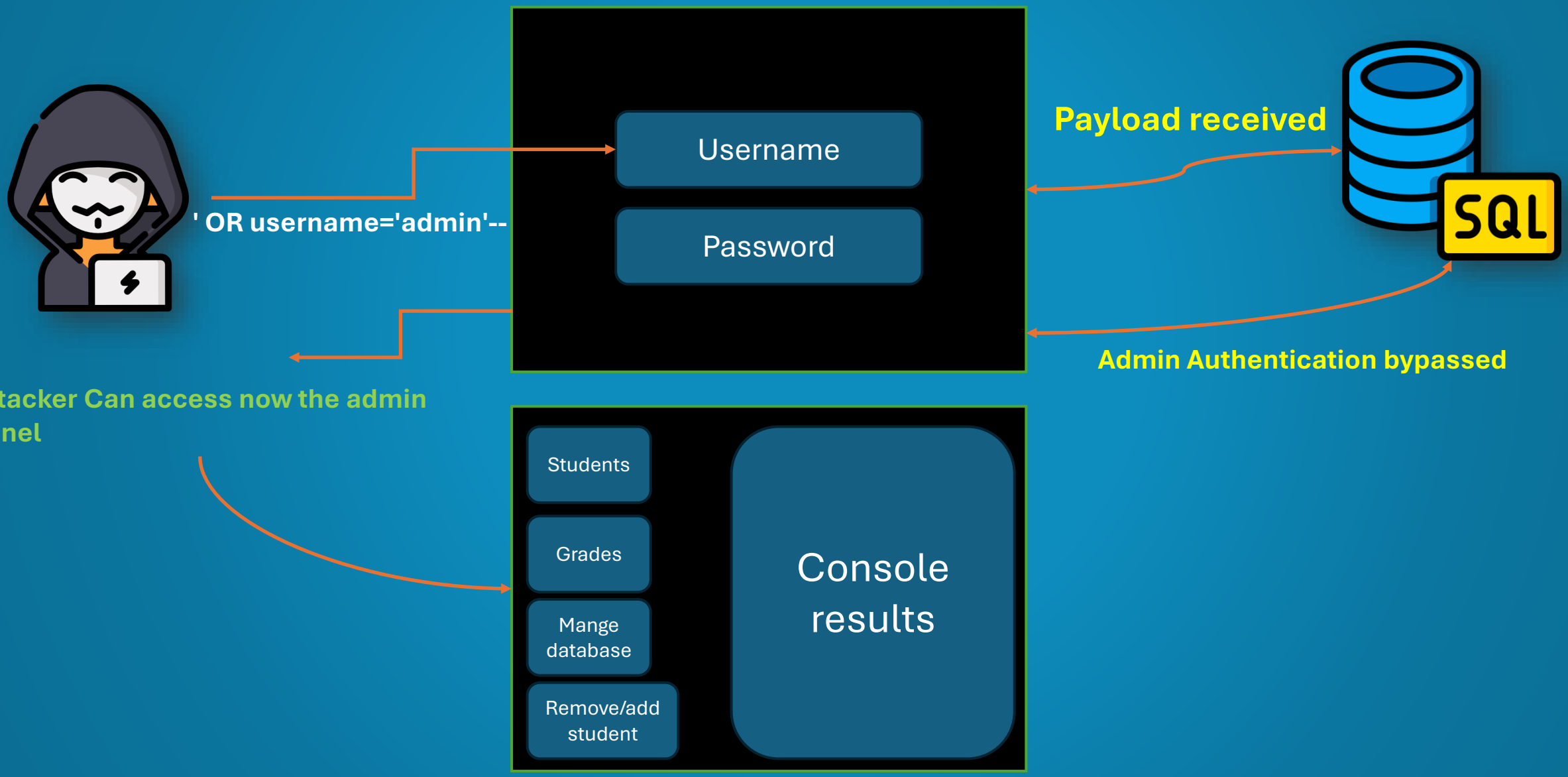
Complete scenario:

An attacker targets the student portal login and abuses unsafe SQL query construction. Because user input is mixed directly into the SQL statement, the attacker may bypass authentication and access accounts without valid credentials.

Fix:

Use parameterized queries / prepared statements

Scenario 1: SQL Injection Login Bypass



Scenario 1: SQL Injection Login Bypass

Fix idea

Use **parameterized queries** instead of building SQL with strings. OWASP recommends prepared statements because they separate SQL code from user-controlled data, and Python's sqlite3 documentation also warns against assembling queries with Python string operations.

Snipp code:

```
# Secure login query using parameterized SQL
def get_user_by_username(db, username):
    return db.execute(
        "SELECT id, username, password_hash, role, mfa_enabled "
        "FROM users WHERE username = ?", # prepared statements applied
        (username,)
    ).fetchone()
```

Scenario 2: Plaintext Password Exposure

Asset:	**User credentials in the SQLite database**
Threat:	**Attacker gains access to database contents**
Vulnerability:	**Passwords are stored without hashing**
Impact:	**Passwords become immediately readable; account takeover becomes easier**
Likelihood:	**Medium**
Risk level:	**High**

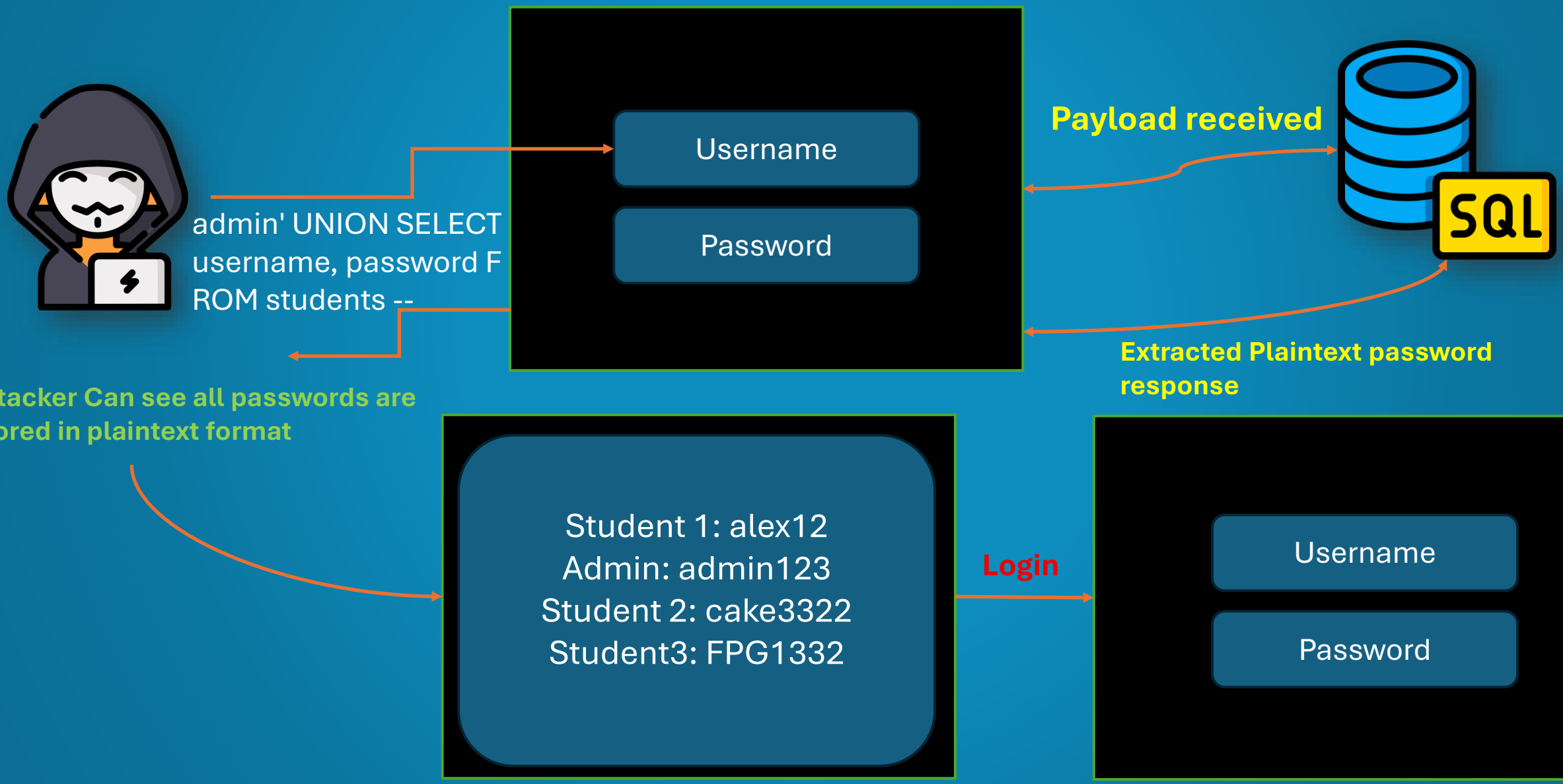
Complete scenario:

If an attacker accesses the SQLite database, plaintext passwords can be read directly. This increases the impact of the database compromise because the attacker may use those passwords to log in to the portal or try them on other services.

Fix:

Hash passwords using a modern password hashing algorithm.

Scenario 2: Plaintext Password Exposure



Scenario 2: Plaintext Password Exposure

Fix idea

Store **password hashes**, not readable passwords. OWASP recommends modern adaptive password hashing such as **Argon2id**, with scrypt, bcrypt, or PBKDF2 as alternatives depending on environment; NIST also says password verifiers should store passwords in a salted, hashed form resistant to offline attacks.

```
from werkzeug.security import generate_password_hash, check_password_hash

def create_user(db, username, password, role):
    password_hash = generate_password_hash(password) # generating password hash

    db.execute(
        "INSERT INTO users (username, password_hash, role, mfa_enabled) "
        "VALUES (?, ?, ?, ?)",
        (username, password_hash, role, 0)
    )
    db.commit()

def verify_password(stored_hash, password):
    return check_password_hash(stored_hash, password)
```

Scenario 3: Missing MFA

Asset:	**Admin account**
Threat:	**Attacker uses stolen credentials**
Vulnerability:	**No MFA required after password entry**
Impact:	**Admin account takeover**
Likelihood:	**High**
Risk level:	**High**

Complete scenario:

An attacker obtains an admin password from the database. Because the portal does not require MFA, the password alone is enough to access the admin account and perform sensitive actions.

Fix:

Require MFA for admin users and sensitive actions

Scenario 3: Missing MFA

admin' UNION SELECT username, password FROM students --

Username

Password

Payload received

Extracted Plaintext password response

Attacker Can see all passwords are stored in plaintext format and can take admin credentials to make normal login

Student 1: alex12

Admin: admin123

Student 2: cake3322

Student3: FPG1332

Login

MFA MISSING

Username

Password

Scenario 3: Missing MFA

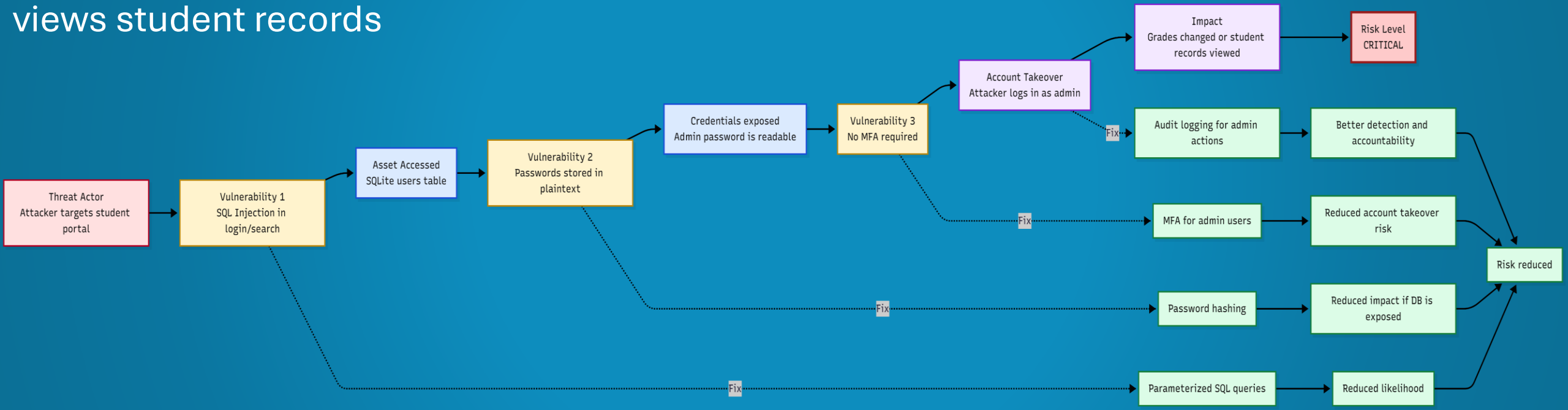
Fix idea

Require an extra factor for admin users. OWASP defines MFA as requiring more than one type of evidence to authenticate, such as something you know plus something you have. NIST also lists OTP and multi-factor authenticators as authentication methods.

```python
import secrets
# Generate OTP code for the admin when every time he want to login
def generate_lab_otp():
    # Classroom demo OTP, not production MFA
    return f"{secrets.randbelow(1_000_000):06d}"

def require_admin_mfa(user, submitted_otp, session):
    if user["role"] != "admin":
        return True

    expected_otp = session.get("lab_admin_otp")

    if not expected_otp:
        session["lab_admin_otp"] = generate_lab_otp()
        return False

    return submitted_otp == expected_otp
```

Scenario 4: Chained Risk Scenario

Attacker finds SQL injection, Attacker accesses user data, Passwords are visible in plaintext, No MFA blocks the attacker, Attacker logs in as admin, Attacker changes grades or views student records



Complete scenario:

An attacker exploits SQL injection in the student portal to access the SQLite users table. Because passwords are stored in plaintext and MFA is not enabled, the attacker logs in as an admin and changes grades, causing privacy breach, integrity damage, and reputational loss.

How Scenarios Improve Risk Analysis Accuracy ?

Scenarios improve accuracy because:

1. Gives a clear view about what are the real assets that effected from the attack
2. Tells who is the cause behind this event which is here the threat actor (attacker
3. Where was the weakness persists in the system that allows the event to occur
4. What is the real impact that effect the system when event occurred
5. How this likely this risk will occur in this system
6. What type of fixing operation reduce this risk

Answers to Required Questions

What makes a strong risk scenario?

We call scenario is strong when it realistic and take all sides inside the environment not only taking one side and left the others like focusing in the software and leaving the hardware updates , it must use measurement and provide clear view about the assets , threats , vulnerabilities in the system, calculate risk level , impact and likelihood and provide fix's.

Why do students often confuse threat and vulnerability?

Because of there appearance in not clear context so this makes them looks the same but in reality they are not, the vulnerability is the entry point weakness to our system, threat is the cause or the someone how use these entry point (weakness) to access our system assets or cause damage.

How can scenarios improve risk analysis accuracy?

They connect technical issues to real consequences. This prevents the unclear view of analysis and helps students judge likelihood, impact, and priority more accurately.

References

NIST SP 800-30 Rev. 1 — Guide for Conducting Risk Assessments

NIST CSRC Glossary — Risk and Threat definitions

NIST SP 800-63B — Digital Identity Guidelines

OWASP SQL Injection Prevention Cheat Sheet

OWASP Password Storage Cheat Sheet

MITRE CWE-256 — Plaintext Storage of a Password

Open FAIR — Loss scenario and risk analysis model

THANK YOU